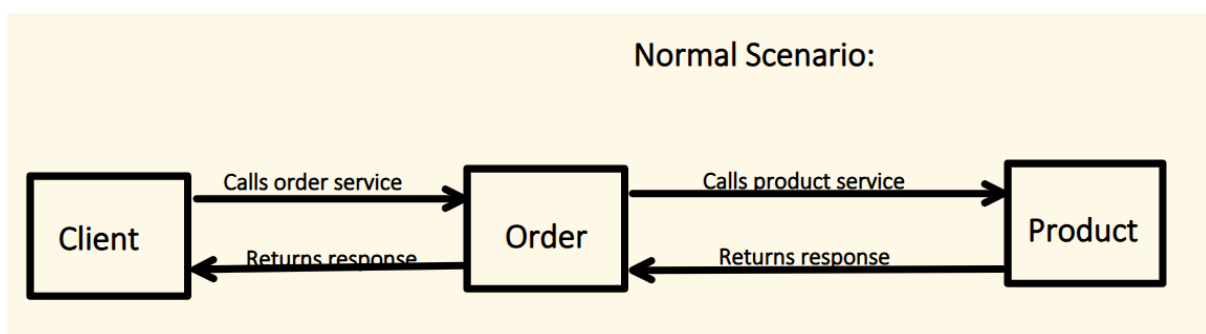


What is Fault-Tolerant Microservice?

- Fault Tolerant Microservice is a service, which continues to work even when downstream system fails.
- Instead of crashing or Cascading failures, it handles the failure gracefully.

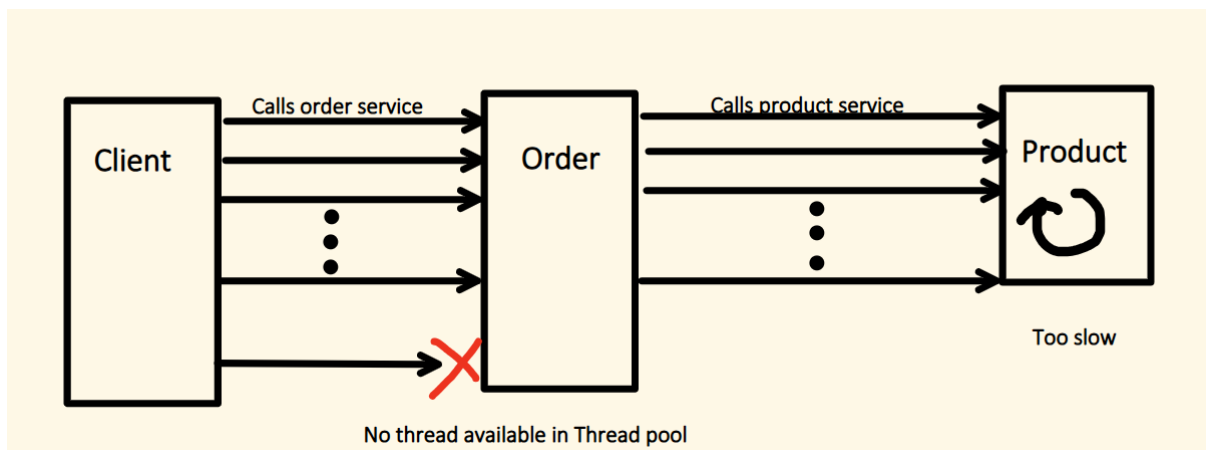
What its required?

- As in microservices, different services communicate over the network.
- And its possible that 1 service unavailability could bring down the whole system.



Lets say because of either buggy code or Infrastructure issue "Product" service becomes too slow:

- Say, product service become slow and taking 60 seconds to respond.
- If proper fault tolerant is not applied, then a call from Order service will hang for 60sec.
- Thus, a thread is blocked for that period.



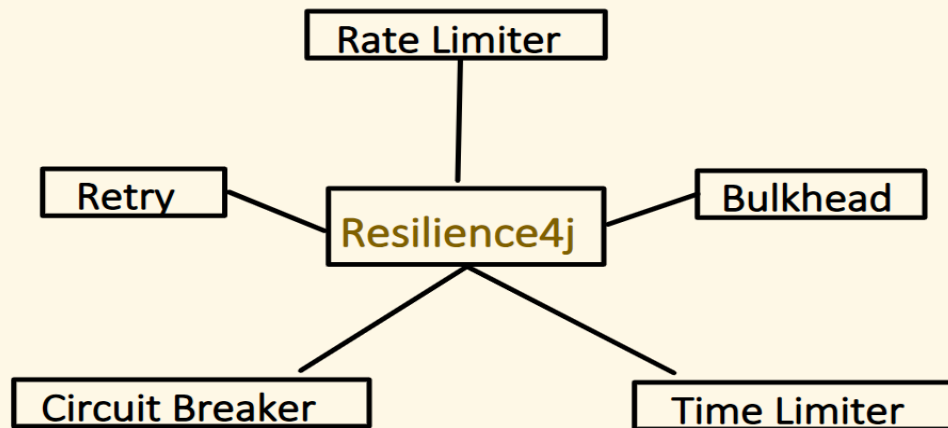
- if sudden burst of request comes from Order to Product service, all threads will now get blocked (waiting for response).
- And because of this, clients of Order service also get blocked.
- Eventually, services will run out of threads and start rejecting the request.

This is know as CASCADING FAILURE.

means, issue is at Product service, but now propagated to other services too.

To avoid this, we need to build Fault tolerant microservice.

And here comes:



Lets go in depth one by one in recommended logical ordering:

RateLimiter → Bulkhead → TimeLimiter → CircuitBreaker → Retry

Why Ordering required?

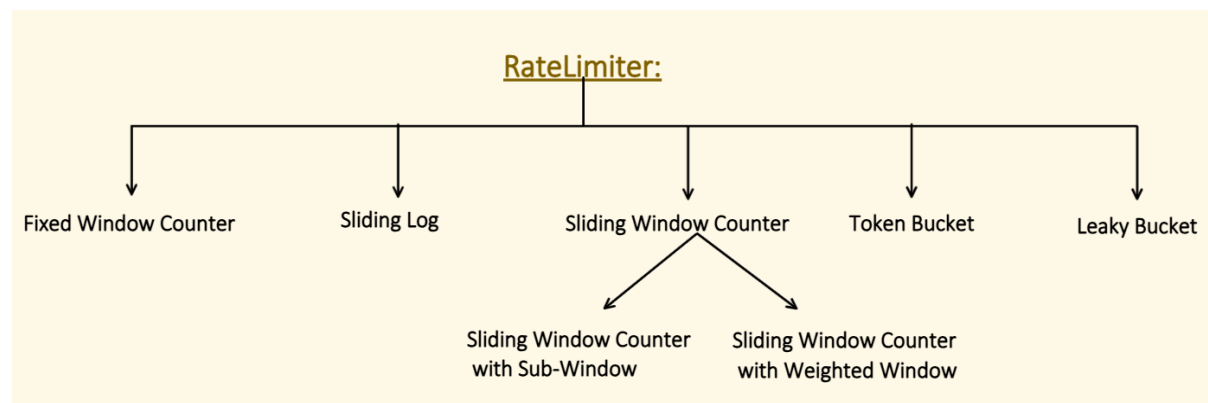
For example: Always apply RateLimiter before Retry, so that retry logic doesn't overwhelm the system with already limited traffic.

Else we end up in retry the traffic, which will get blocked by RateLimiter later.

RateLimiter:

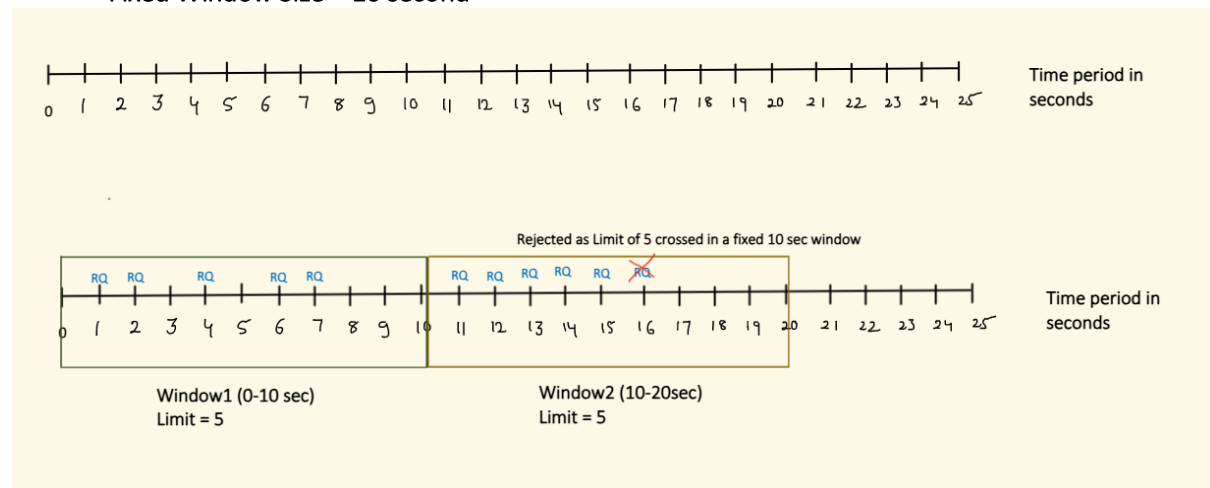
It controls the number of requests allowed to a microservice in a given time window.

So in other words, it protect our system from sudden traffic spike (like in DDOS attack).



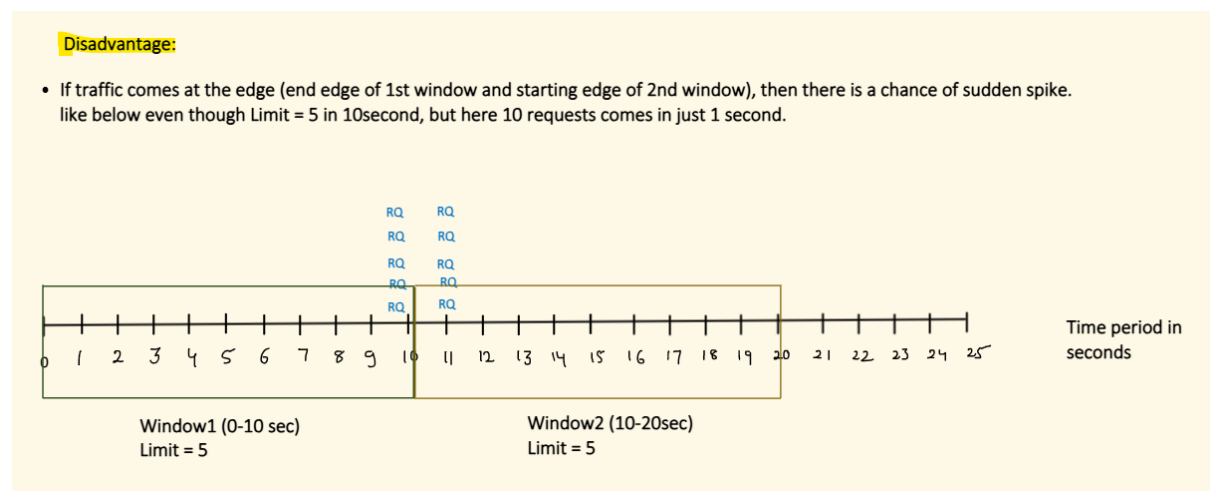
Fixed Window Counter:

- Counts how many request happens in a fixed window.
- If the count crossed the Limit, then reject the request.
- For example:
- Limit = 5
- Fixed Window Size = 10 second



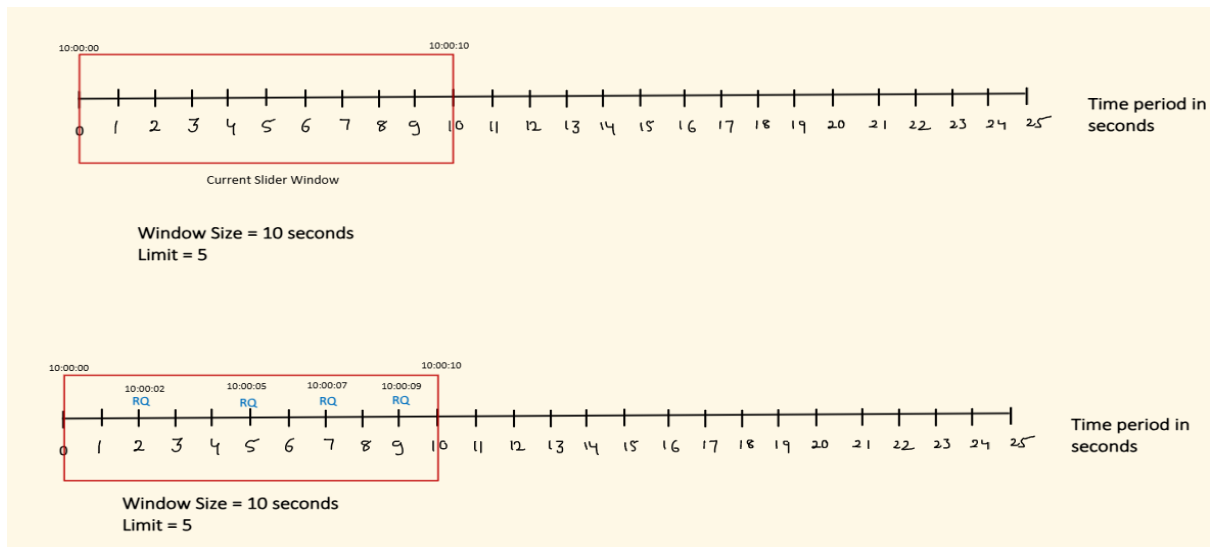
Disadvantage:

- If traffic comes at the edge (end edge of 1st window and starting edge of 2nd window), then there is a chance of sudden spike. like below even though Limit = 5 in 10second, but here 10 requests comes in just 1 second.

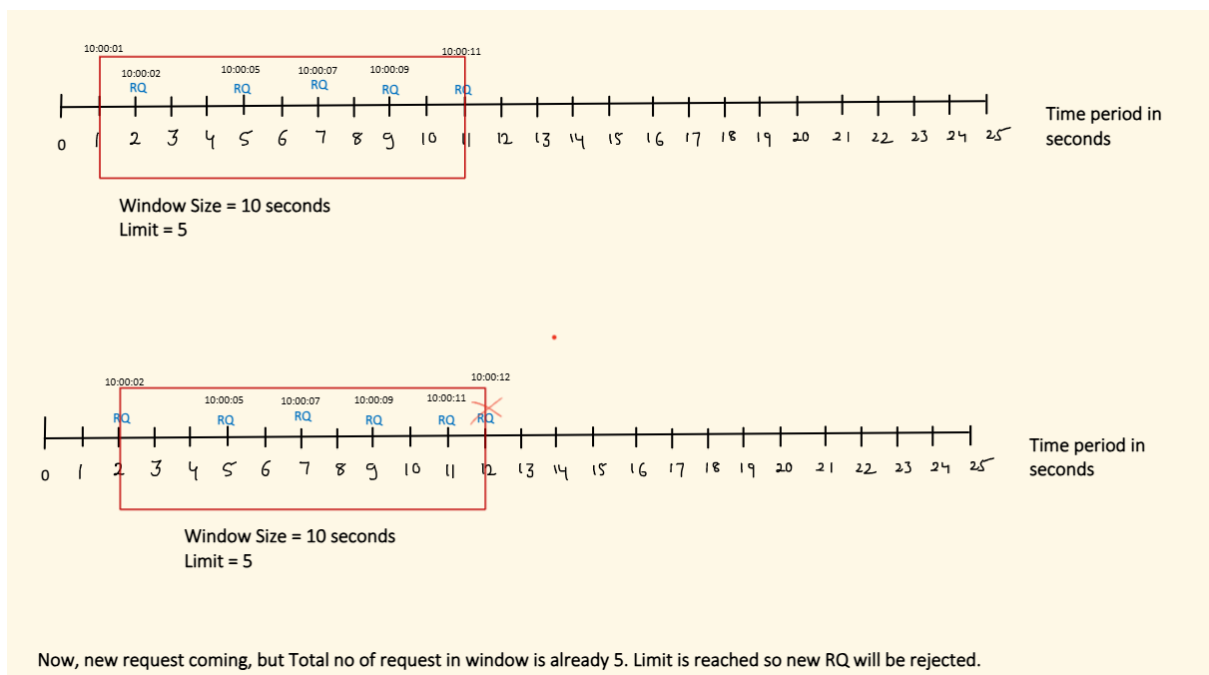


Sliding Log:

- For each accepted request, it stores the exact timestamp.
- For each request comes in, it first check, how many request happened in last X seconds (Window size).
- If no of requests reach the LIMIT, it will reject the request.

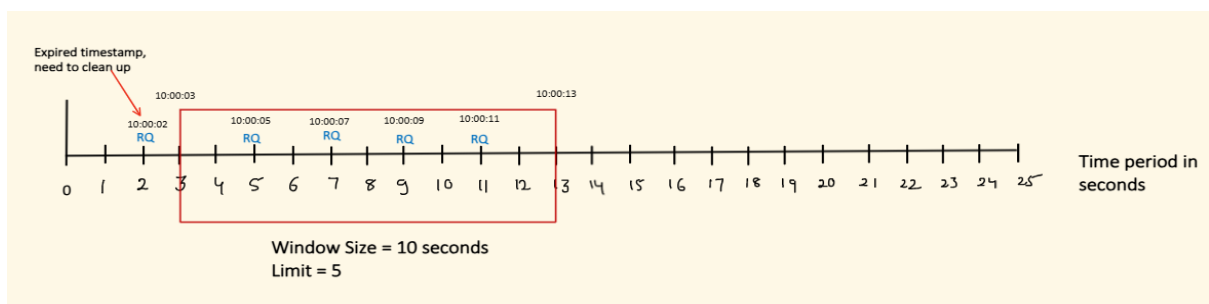


Now, when window size crossed, slider moves on.
So it consider only those request in count, which are in slider window only.



Disadvantage:

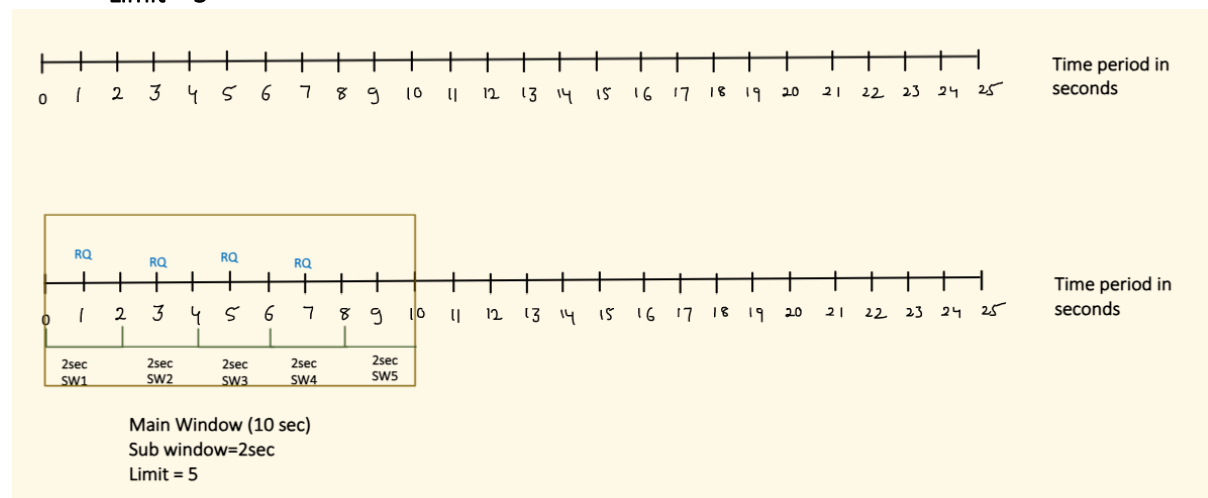
- Need to maintain timeframe info for each accepted request. Which needs more memory.
- Extra overhead of clean up of expired timestamps.



Sliding Window Counter:

Sliding Window Counter with Sub-Window:

- Main window is divided into equal sized sub-windows.
- Keep Tracks of the number of request in each sub-window.
- To decide if request need to be rejected or not, it consider only active sub-windows.
- Main window slides(by sub-window size) over time.
- Window Size = 10 seconds
- Sub-Window = 2 seconds
- Limit = 5



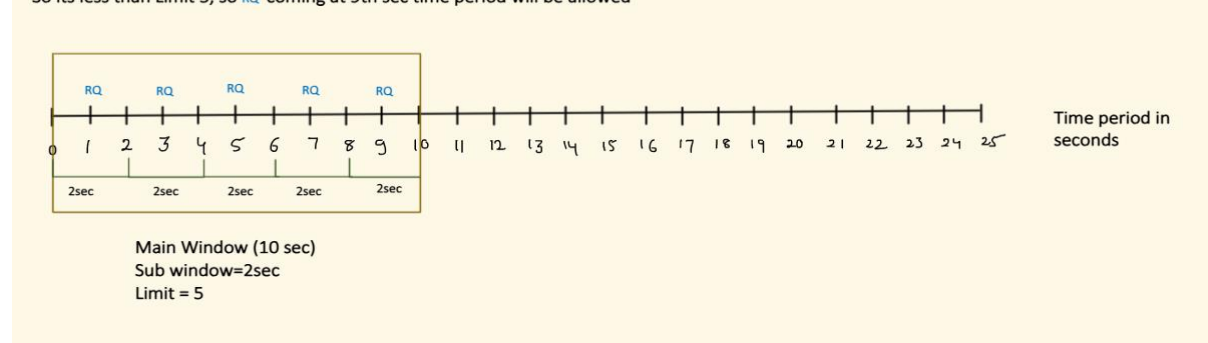
Say for new Request **RQ** coming at 9th sec time period:

It will sum up the requests in all active Sub-windows:

$$SW1(0\text{sec}-2\text{sec}) + SW2(2\text{sec}-4\text{sec}) + SW3(4\text{sec}-6\text{sec}) + SW4(6\text{sec}-8\text{sec}) + SW5(8\text{sec}-10\text{sec})$$

$$1 + 1 + 1 + 1 + 0 = 4$$

So its less than Limit 5, so **RQ** coming at 9th sec time period will be allowed

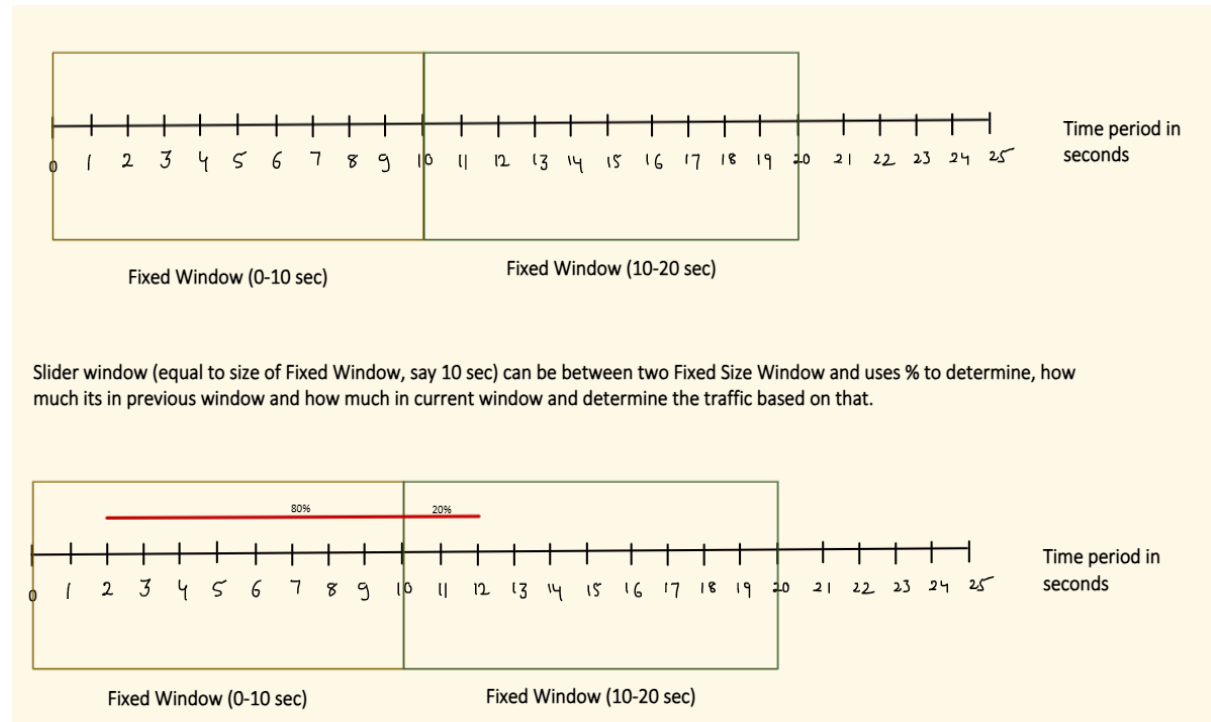


Disadvantage:

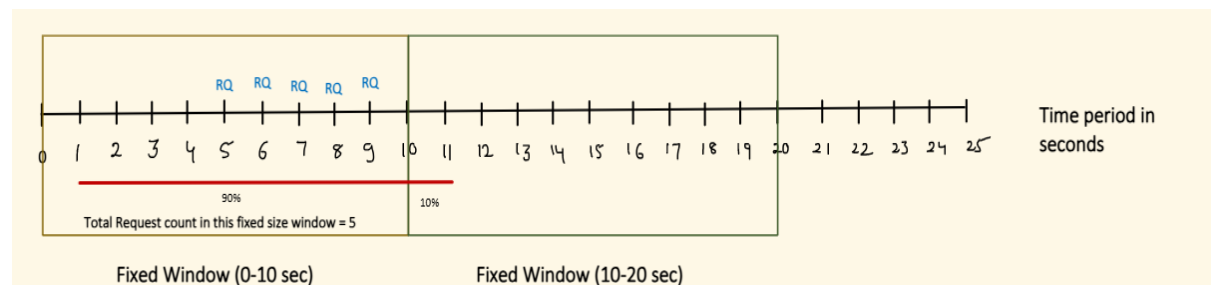
- More complex than Fixed window logic.
- Need to maintain Sub window info and request count per sub window. Which needs more memory.

Sliding Window Counter with Weighted Window:

- It tries to simplify the logic by removing Sub-window granularity and bringing some approximation.
- It combined Fixed Window + Sliding Log Concept.



Let's Say, Limit = 5 in a slide window.



Now lets say **RQ** is coming at 11th second of time period.

Now, it uses a formula to compute, how much request from each window it should consider.

Total Requests in Current Fixed Window + 90% of Total Request in previous window (i.e. 5)
(using 90% based on above example shown, it can be different)

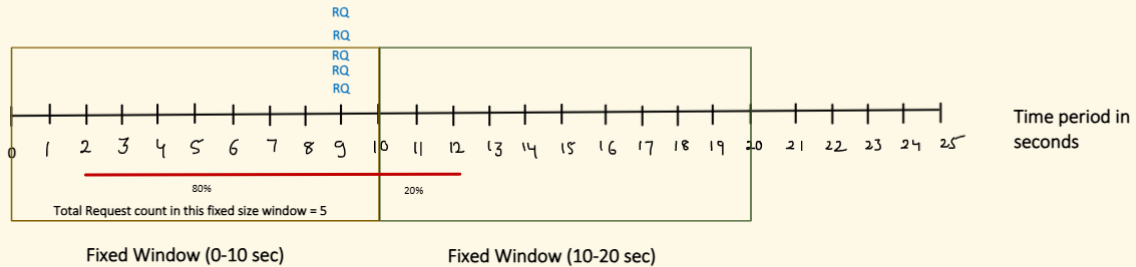
Total Requests in Current Fixed Window (10-20sec) = 0
90% of Total Request in previous window (i.e. 5) = 4.5 say we Ceil/round it to ~5

0 + ~5 = 5

Limit is reached of the slider window so Request is Rejected.

Disadvantage:

- It assume uniform distribution of traffic, but that might not be true sometimes.



Now, new RQ coming at 12th second

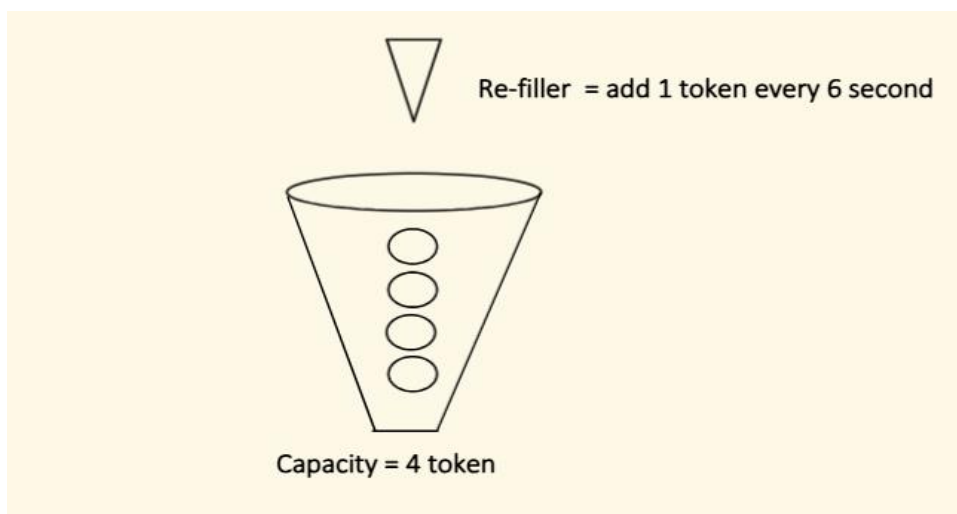
Total Requests in Current Fixed Window + 80% of Total Request in previous window (i.e. 5)
(using 80% based on above example shown, it can be different)

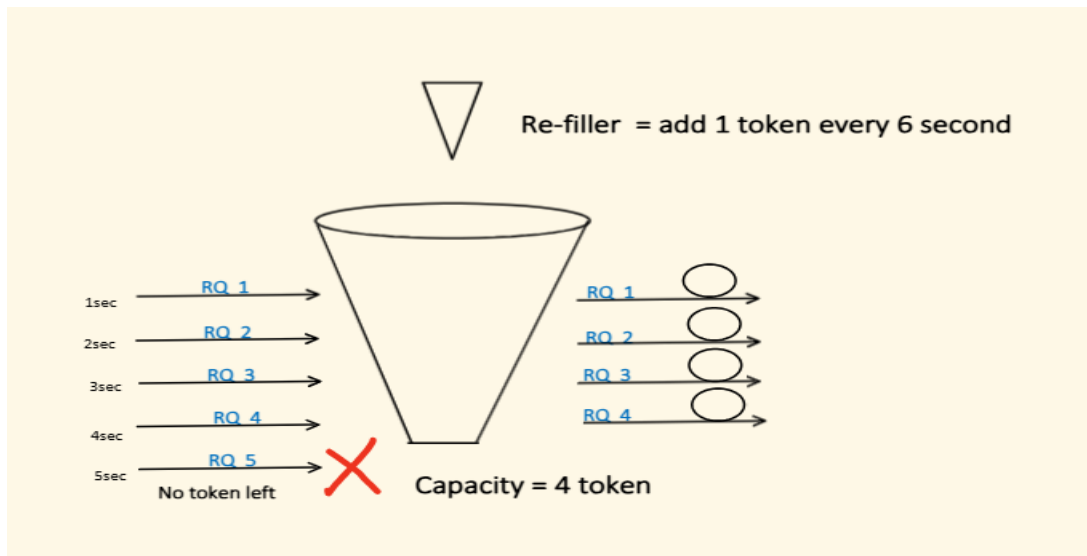
$$0 + 4 = 4$$

4 is less than Limit 5, so request is permitted

Token Bucket:

- Limited Tokens are placed in a bucket.
- Each request consume one token.
- If there is no token left, request is denied.
- There is a re-filler, which adds the token in bucket at regular time interval.
- If Bucket capacity is full and can not hold more token, token overflow will happen and token will be rejected.





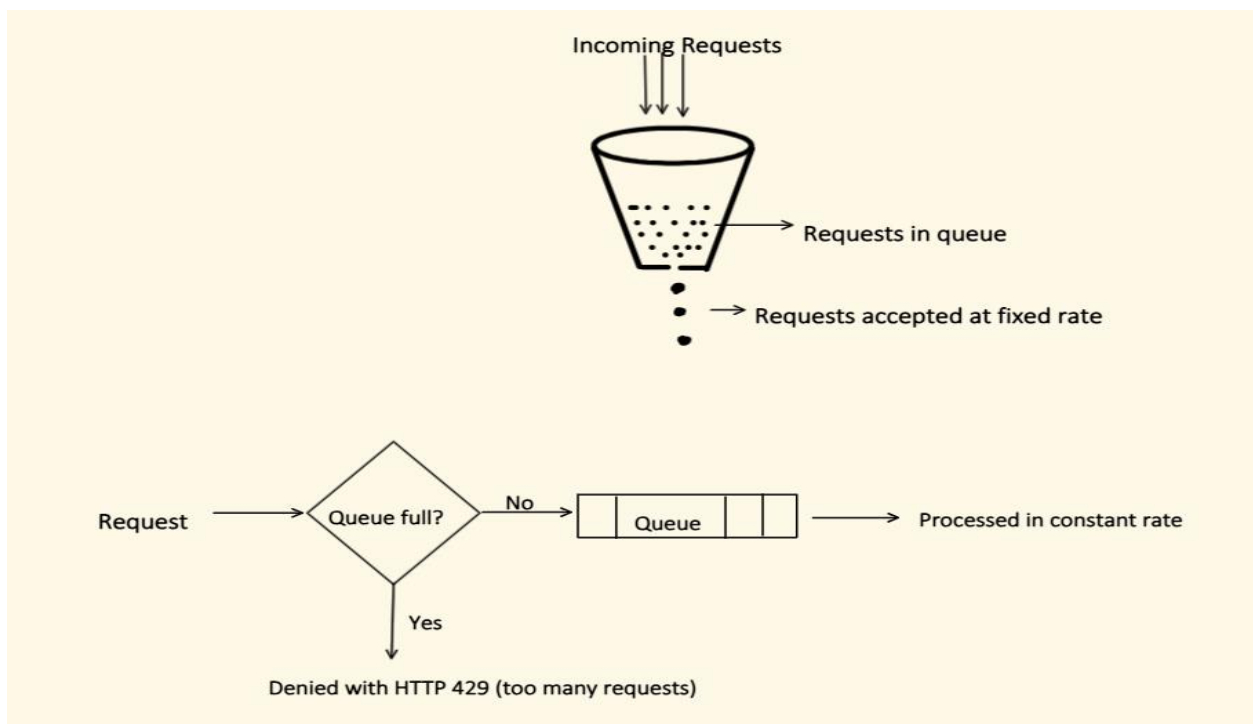
Disadvantage:

- A quick burst is possible if token capacity is not set properly.

Let's say system remains idle for some time and token keep on accumulating, and when sudden burst comes, it will be allowed as token is present.

Leaky Bucket:

- Requests are Queued and accepted at fixed rate.
- If queue is full, incoming request will overflow



Disadvantage:

- Latency increases, as request might have to wait in queue for its turn.
- Queue size is a bottle neck. So size should be properly set.

Implementation in Spring boot:

Pom.xml dependency

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.1.0</version>
</dependency>
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}
```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

Internally it uses AOP functionality, that's why the method on which this @RateLimiter annotation applies, need to be public and Spring managed bean.



```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @RateLimiter(name = "productRateLimiter", fallbackMethod = "rateLimitedFallback")
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }

    public void rateLimitedFallback( String id, Throwable t) {
        System.out.println("Rate limit exceeded. Try later");
        //throw exception here and handle it gracefully
    }
}
```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#bucket filled with 2 tokens every 10second,
#if there is no token wait for 1sec before rejecting the request
resilience4j.ratelimiter.instances.productRateLimiter.limitForPeriod=2
resilience4j.ratelimiter.instances.productRateLimiter.limitRefreshPeriod=10s
resilience4j.ratelimiter.instances.productRateLimiter.timeoutDuration=1s
```

Fallback method, return type and parameter(Additionally only Throwable need to add more) should match with the original method, else default fallback method provided by RateLimiter framework will get invoked.

Property	Meaning
limitForPeriod	Number of requests allowed
limitRefreshPeriod	Time window
timeoutDuration	How long request waits for permission

This means:

Allow only 2 calls every 10 seconds

After 2 requests: Request gets rejected

because: `timeoutDuration = 1` (Wait up to 1 second for permission)

After waiting: Timeout exceeded

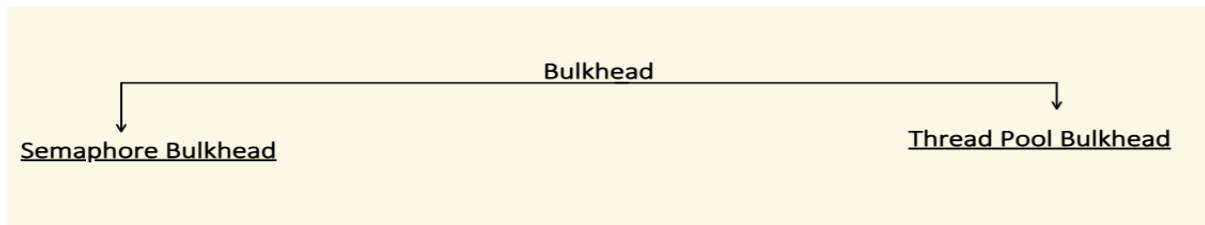
Then: **Fallback will be called.**

Resilience4j Internally, its `RateLimiter` uses a: **Atomic Cycle-Based Rate Limiting Algorithm**

which behaves very similar to a: **Fixed Window Rate Limiter**

Bulkhead: Fault-Tolerant Microservice (Part-2)

It helps to control **how many concurrent requests can go to downstream service.**



1. Semaphore Bulkhead

Limits concurrent calls using semaphores.

No separate thread pool.

- Limits the number of concurrent calls using counter.
- If the limit is reached, further calls are rejected immediately or blocked for specific wait time.

it limits the number of concurrency call, via **Semaphore Lock**.

Example

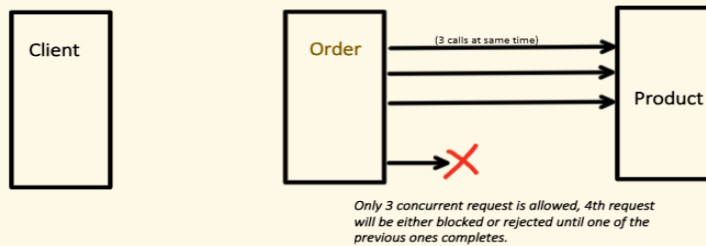
Allow only 3 concurrent requests

4th request:

- rejected immediately
- or waits (configurable)

Ex: let say our application is "Order"

Use-Case: Let say, "Product" is small and light weight service and can only handle 3 concurrent request.



Above use case is resolved through "Semaphore Bulkhead"

- It also protects **our application from our downstream services** by limiting how many threads we allocate to them.

Pom.xml dependency

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.1.0</version>
</dependency>
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}
```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Bulkhead(name = "productService", type = Bulkhead.Type.SEMAPHORE, fallbackMethod = "productFallback")
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
        System.out.println("Response from Product api call is: " + response);
    }

    public void productFallback( String id, Throwable t) {
        System.out.println("too many concurrent request, pls try again later");
        //throw exception here and handle it gracefully
    }
}
```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#only 2 concurrent calls allowed and
#for further req, if there is no thread available, request will fail immediately
resilience4j.bulkhead.instances.productService.maxConcurrentCalls=2
resilience4j.bulkhead.instances.productService.maxWaitDuration=0
```

Accepted values for maxWaitDuration: It accepts values in the following format:

<number><time-unit>

Supported time units:

- ms — milliseconds
- s — seconds
- m — minutes
- h — hours
- d — days (not typical, but valid)

Example:

```
resilience4j.bulkhead.instances.productService.maxWaitDuration=0    # Reject immediately
resilience4j.bulkhead.instances.productService.maxWaitDuration=300ms # Wait 300 milliseconds
resilience4j.bulkhead.instances.productService.maxWaitDuration=2s   # Wait 2 seconds
resilience4j.bulkhead.instances.productService.maxWaitDuration=1m   # Wait 1 minute
```

2. Thread Pool Bulkhead

Uses separate thread pool + queue.

- Assign a dedicated thread pool for each service.
- Only that pool is used to call the downstream.

Use-Case: Let say our Order Service has 2 APIs (endpoints)

API 1 : make a downstream call to Product service.

API 2 : make a downstream call to Payment service.

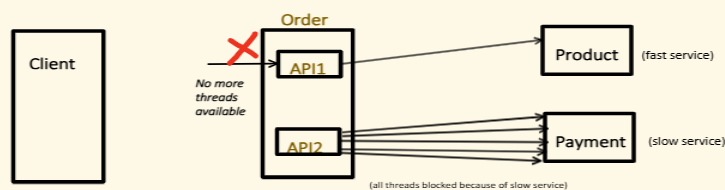
Product service takes ~100 milliseconds to process a request (Fast Service)
and

Payment service takes ~5second to process a request (Slow service)

What if there is a sudden spike in **API 2** traffic and more and more calls are made to Payment service (say calls are not concurrent).

Since Payment service is slow, it holds or blocks the thread longer (for ~5seconds) and its possible that all the threads of Order service now blocked (by Payment service only).

Now even the Order service fast endpoint (**API 1**) becomes unavailable as there is no threads available.



Above use case is resolved through "Thread Pool Bulkhead"

- The AOP proxy intercepts your method call.
- It submits your method to the Bulkhead's thread pool (configured via application.properties).

```
CompletableFuture.supplyAsync(() -> {
    return ourMethodLogic(); // The whole method body runs here
}, bulkheadThreadPoolExecutor);
```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Bulkhead(name = "productService", type = Bulkhead.Type.THREADPOOL, fallbackMethod = "productFallback")
    public CompletableFuture<String> invokeProductAPI(String id) {
        return CompletableFuture.completedFuture(productClient.getProductById(id));
    }

    public CompletableFuture<String> productFallback(String id, Throwable t) {
        //even though its async, we need to properly handle the exception
        //else our client will get 500 response with BulkheadFullException
        return CompletableFuture.completedFuture(value: "Product Service is busy");
    }
}
```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#max 3 threads in the pool and that's the max pool size too.
#if all 3 threads are busy, max 2 more requests will be put into queue
#from 6th request onwards, it will start rejecting the requests.
resilience4j.thread-pool-bulkhead.instances.productService.coreThreadPoolSize=3
resilience4j.thread-pool-bulkhead.instances.productService.maxThreadPoolSize=3
resilience4j.thread-pool-bulkhead.instances.productService.queueCapacity=2
```

ThreadPool for productService will be created with these configurations.

Difference Between Semaphore vs ThreadPool Bulkhead

Feature	Semaphore Bulkhead	ThreadPool Bulkhead
Thread isolation	No	Yes
Separate thread pool	No	Yes
Lightweight	Yes	Slightly heavier
Best for	Fast APIs	Slow/blocking APIs
Queue support	No	Yes

TimeLimiter : Fault-Tolerant Microservice (Part-3)

- A **TimeLimiter** is used to limit how long an operation is allowed to run.
- If the operation takes longer than the configured timeout:
- Request is cancelled/timed out
- and fallback can be executed.

Why TimeLimiter is Needed

In distributed systems, external services may become:

- slow
- unresponsive
- partially down

Without timeout handling:

Threads keep waiting

This causes:

- thread pool exhaustion
- increased latency
- cascading failures

Important Point

TimeLimiter is mainly used for:

Asynchronous calls

like:

- CompletableFuture
- async APIs
- non-blocking operations

Configuration

```
</> YAML
resilience4j:
  timelimiter:
    instances:
      paymentTimeLimiter:
        timeoutDuration: 2s
        cancelRunningFuture: true
```

Meaning

Property	Meaning
timeoutDuration	Maximum execution time
cancelRunningFuture	Cancel task after timeout

Service

```
</> Java
@Service
public class PaymentService {

    @TimeLimiter(
        name = "paymentTimeLimiter",
        fallbackMethod = "fallback"
    )
    public CompletableFuture<String> processPayment() {

        return CompletableFuture.supplyAsync(() -> {

            sleep(5000);

            return "Payment Success";
        });
    }

    public CompletableFuture<String> fallback(Exception ex) {

        return CompletableFuture.completedFuture(
            "Payment Timeout"
        );
    }
}
```

Short Definition

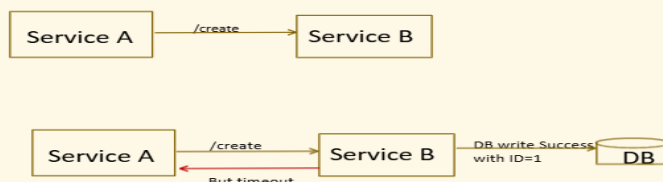
"TimeLimiter in Resilience4j is used to limit how long an asynchronous operation can run. If execution exceeds the configured timeout, the operation is cancelled and fallback logic can be executed."

Retry : Fault-Tolerant Microservice (Part-4)

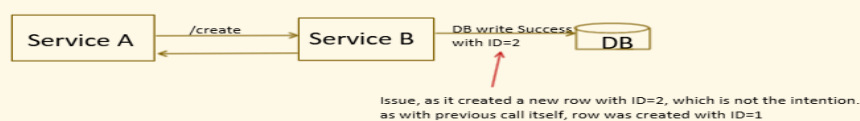
- In distributed system, call to downstream service might fail because of Transient issues.
- **Transient issues** means short temporary issues like network issue, timeouts etc.
- And retry of same request after a short delay can get succeed

When to retry?

- Do not retry, if operations is non-idempotent.
Idempotency make sure that, if request is retried multiple times, duplicate processing will not happen
For e.g.,



What if user retry in this case, as timeout happens but this endpoint is not idempotent



. Do not retry on permanent failures :

e.g., 4xx errors

4xx are validation errors, means there is some issue with input request.

So even you retry with same request, it will fail again in the validation error.

. Rule of Thumb of this is, Retry on 5xx errors:

5xx error means, there is some issue with system and request is not successful.

It could be a transient issue.

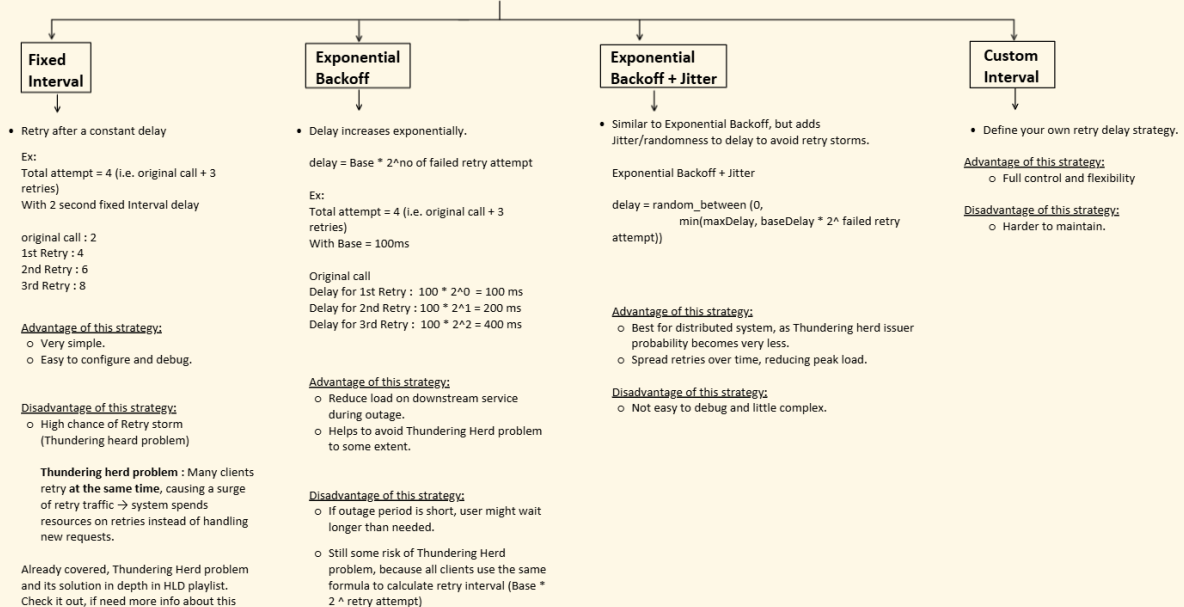
But its pretty safe to retry those errors.

- Retry Network errors and Timeouts only if endpoint ensures Idempotency.

Retry Decision Table

HTTP Status Code	Meaning	Retry?
200 OK	Success	No
400 Bad Request	Client error (invalid input)	No
401 Unauthorized	Authentication needed	No
403 Forbidden	Access denied	No
404 Not Found	Resource not found	No
409 Conflict	Resource not found	No
429 Too Many Requests	Rate limiting	Yes (with Delay)
500 Internal Server Error	Server issue	Yes
502 Bad Gateway	Downstream service failure	Yes
503 Service Unavailable	Service down/overloaded	Yes
Network Error	Connection failure, timeout	Yes (Idempotent only)

Types of Retry



Fixed Interval:

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}
```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @Retry(name = "productService", fallbackMethod = "productFallback")
    public void invokeProductAPI(String id) {
        String response = productClient.getProductById(id);
    }

    public void productFallback(String id, Throwable t) {
        System.out.println("Product Service is busy");
    }
}
```

@Retry Applied on top of method, where only downstream service invocation is present. Be sure that method should contain only that code which you want to retry. When all retry will finish off, then fallback method will be invoked.

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#fixed delay interval
resilience4j.retry.instances.productService.maxAttempts=3
resilience4j.retry.instances.productService.waitDuration=2s
```

application.properties

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#exponential backoff
resilience4j.retry.instances.productService.maxAttempts=4
resilience4j.retry.instances.productService.waitDuration=1s
resilience4j.retry.instances.productService.enableExponentialBackoff=true
resilience4j.retry.instances.productService.exponentialBackoffMultiplier=2
```

initial wait time
default is fixed interval, so we need to tell specifically.
factor

Backoff.java (framework class)

```
try {
    nextBackoff = firstBackoff.multipliedBy((long) Math.pow(factor, (context.iteration() - 1)));
    if (nextBackoff.compareTo(maxBackoffInterval) >= 0) {
        nextBackoff = maxBackoffInterval;
    }
}
```

Formula is:
Initial wait time * (factor ^ no of failed retry attempts)

1st retry = 1000ms * (2 ^ 0) = 1000ms = 1sec delay
2nd retry = 1000ms * (2 ^ 1) = 2000ms = 2sec delay
3rd retry = 1000 * (2 ^ 2) = 4000ms = 4sec delay

Exponential Backoff + Jitter :

```
server.port=8081
spring.application.name=order-service
eureka.client.service-url.defaultZone=http://localhost:8761/eureka

#exponential backoff + jitter
resilience4j.retry.instances.productService.maxAttempts=4
resilience4j.retry.instances.productService.waitDuration=1s
resilience4j.retry.instances.productService.enableExponentialBackoff=true
resilience4j.retry.instances.productService.exponentialBackoffMultiplier=2
resilience4j.retry.instances.productService.enableRandomizedWait=true
```

Need to enable this

Custom Retry

```
@Configuration
public class Config {

    @Bean
    public Retry customRetry() {
        IntervalFunction fibonacciInterval = attempt -> {
            return 2000L; // return milliseconds as long
        };

        RetryConfig config = RetryConfig.custom()
            .maxAttempts(6)
            .intervalFunction(fibonacciInterval)
            .retryExceptions(Exception.class)
            .build();

        return Retry.of( name: "customRetry", config);
    }
}
```

IntervalFunction, defines the wait time between retries.

Within RetryConfig object, we provide all the configs like max attempt, which IntervalFunction to use to compute the delay etc.

Then create Retry object from RetryConfig. We can give any name to our Retry.

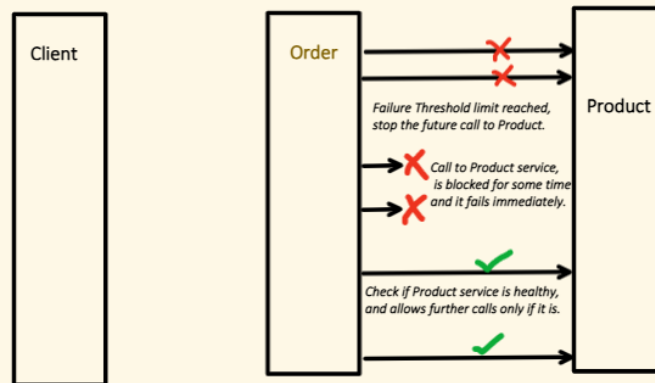
Need to manually wrap an object within Retry. As @Retry annotation works only for those Retry types for which we can provide config in application.properties.

- It unnecessarily adds load to Product service and because of that Product service might take longer time to recover.
- Order service is unnecessarily wasting its resource (latency and thread blocking) by making call that is likely going to fail.

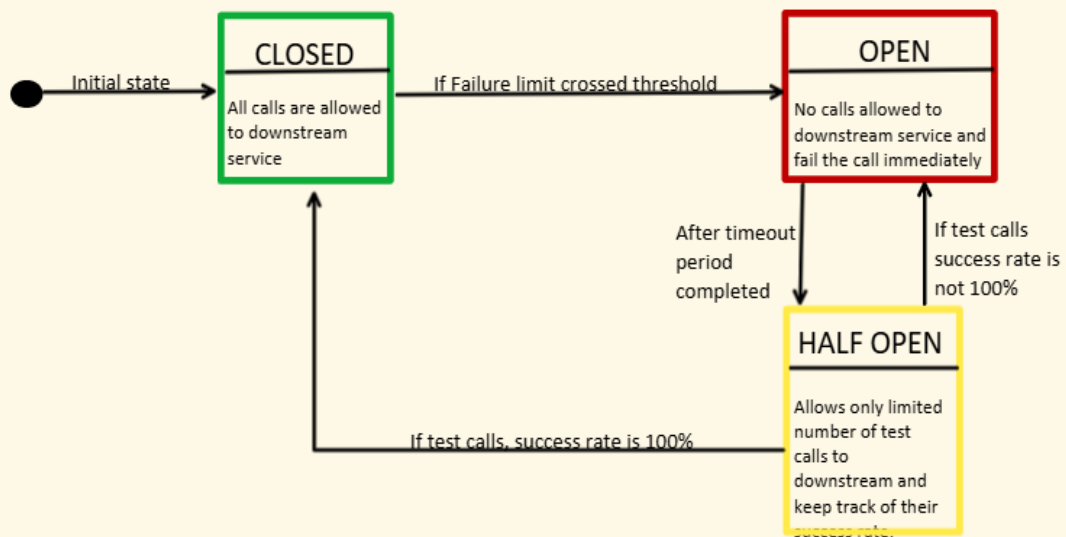
So, what's the solution?
How does Order service know, when to **stop** the downstream Product service call and when to **restart**?

Answer is "Circuit Breaker", again, lets revisit its definition.

It prevents an application to make repeated calls to a downstream service that is likely to fail.



States of Circuit Breaker:



Pom.xml dependency

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
  <version>2.1.0</version>
</dependency>
```

```
@RestController
@RequestMapping("/orders")
public class OrderController {

    @Autowired
    OrderService orderService;

    @GetMapping("/{id}")
    public void callProductAPI(@PathVariable String id) {
        orderService.invokeProductAPI(id);
    }
}
```

```
@FeignClient(name = "product-service")
public interface ProductClient {

    @GetMapping(value = "/products/{id}")
    String getProductById(@PathVariable("id") String id);
}
```

```
@Component
public class OrderService {

    @Autowired
    ProductClient productClient;

    @CircuitBreaker(name = "productService", fallbackMethod = "fallback")
    public void invokeProductAPI(String id) {
        productClient.getProductById(id);
    }

    public void fallback(Throwable ex) {
        System.out.println("not able to invoke product service");
    }
}
```

application.properties

```
1 server.port=8081
2 spring.application.name=order-service
3 eureka.client.service-url.defaultZone=http://localhost:8761/eureka
4
5 #product service - circuit breaker configurations
6 resilience4j.circuitbreaker.instances.productService.sliding-window-type=COUNT_BASED
7 resilience4j.circuitbreaker.instances.productService.sliding-window-size=10
8 resilience4j.circuitbreaker.instances.productService.minimum-number-of-calls=5
9 resilience4j.circuitbreaker.instances.productService.failure-rate-threshold=50
10 resilience4j.circuitbreaker.instances.productService.wait-duration-in-open-state=10s
11 resilience4j.circuitbreaker.instances.productService.permitted-number-of-calls-in-half-open-state=3
12 resilience4j.circuitbreaker.instances.productService.automatic-transition-from-open-to-half-open-enabled=true
```

sliding-window-type=COUNT_BASED
sliding-window-size=10
Tracks N (in this case 10) number of recent calls.

sliding-window-type=TIME_BASED
sliding-window-size=10s
Tracks calls made in a last N time duration(in this case 10sec)

minimum-number-of-calls = minimum no of call required in a window, before it start evaluating the failure rate.

failure-rate-threshold = circuit moves to OPEN state, once this threshold % limit reached.

wait-duration-in-open-state= time circuit stays in OPEN state, before it moves to HALF_OPEN state (automatically, if automatic-transition-from-open-to-half-open-enabled is set to true)

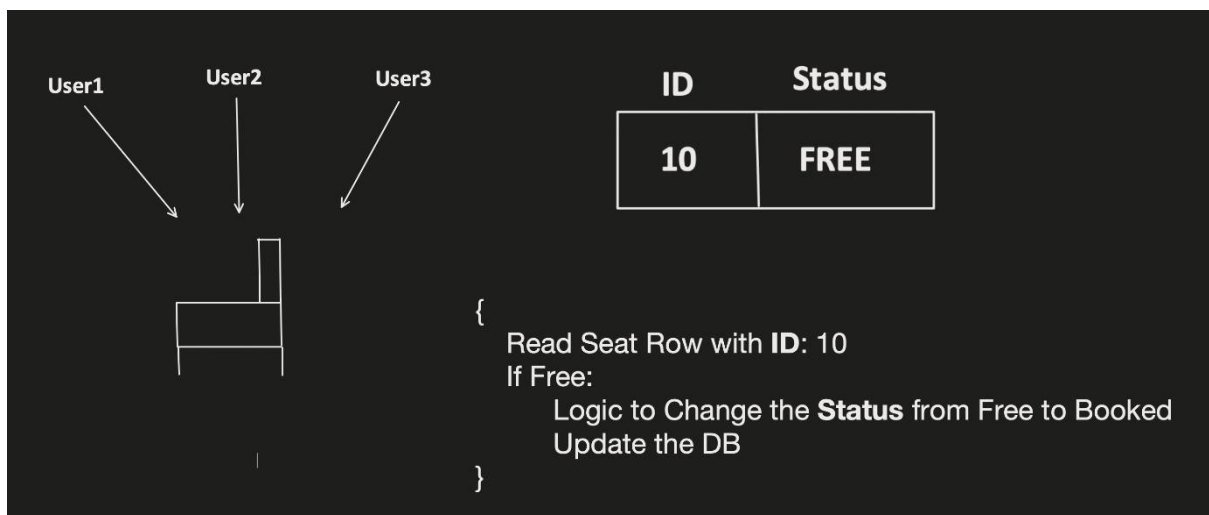
permitted-number-of-calls-in-half-open-state = no of test calls allowed in HALF_OPEN state to find out if downstream is healthy or not.

By default, the Circuit Breaker records all **RuntimeExceptions** and **Errors** as failures.
But if we want specific exception to be recorded and ignored, we can also configured it like below:

```
resilience4j.circuitbreaker.instances.productService.record-exceptions=java.io.IOException,org.springframework.web.client.HttpServerErrorException
resilience4j.circuitbreaker.instances.productService.ignore-exceptions=java.lang.IllegalArgumentException
|
```

Concurrency Control in Distributed Systems

Scenario: Many concurrent request tries to book same Movie theatre Seat



1. Using "**SYNCHRONIZED**" for the Critical Section:

```
synchronized ()
{
  Read Seat Row with ID: 10
  If Free:
    Logic to Change the Status from Free to Booked;
    Update the DB;
}
```

ID	Status
10	FREE

Will this solution works for Distributed System?

Process vs Thread =>

Process:- It's like a one request.

A **process** is an independent program in execution.

Thread :- Each request we can process it using multiple threads.

A **thread** is a smaller unit of execution **within a process**.

When you run **multiple instances of the same service** in a distributed system and those instances handle **transactions**, the main challenge is **data consistency** and **race conditions**.

◆ What Happens with Multiple Service Instances?

Suppose you have:

- 3 instances of PaymentService
- All connected to the same database
- Multiple users making transactions simultaneously

Example:

PaymentService-1

PaymentService-2

PaymentService-3

Now imagine two requests arrive at the same time:

User A withdraws ₹500

User B withdraws ₹700

Both access the same account balance.

🔗 Problem: Race Condition

Assume account balance = ₹1000

Without proper transaction handling:

Step	Instance 1	Instance 2
Read Balance	1000	1000
Calculate	$1000 - 500 = 500$	$1000 - 700 = 300$
Update DB	500	300
Final balance becomes ₹300 instead of negative or rejected.		

This is called a **race condition** or **lost update problem**.

◆ Why Multiple Instances Increase the Problem

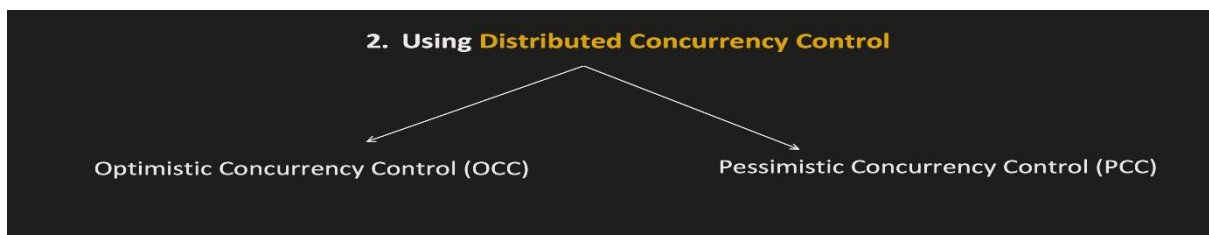
With one instance:

- Requests are easier to control

With multiple instances:

- Requests run truly in parallel
- Each instance has its own memory
- synchronized keyword works only inside one JVM
- One instance cannot control another instance directly

So application-level locking becomes insufficient.



But before we learning this, we **MUST** know below **IMPORTANT** things first.

1. What is the usage of Transaction?
2. What is DB Locking?
3. What are the Isolation Level present?

1. What is the usage of Transaction?

Transaction helps to achieve **INTEGRITY**. Means it help us to avoid **INCONSISTENCY** in our database.

For example:

Debit the Money from A and Credit the money to B

BEGIN_TRANSACTION:

- Debit the Money from A
 - Credit the Money to B
- If all success:
- COMMIT;
- else:
- ROLLBACK;

END_TRANSACTION;

2. What is DB Locking?

DB Locking, help us to make sure that no other transaction update the locked rows.

Lock Type	Another Shared Lock	Another Exclusive Lock
Have Shared Lock	Yes	NO
Have Exclusive Lock	NO	NO

In databases, **shared locks** and **exclusive locks** are used to control concurrent access to data and maintain consistency during transactions.

They are part of database concurrency control in Database Management Systems.

◆ Shared Lock (S Lock)

A **shared lock** allows:

- Multiple transactions to **read** the same data simultaneously
- But prevents any transaction from **modifying** that data while the lock exists (We can't apply Exclusive lock while having Shared lock)

👉 Used mainly for **READ operations**

Example

Transaction T1:

```
SELECT * FROM account WHERE id = 1;
```

Database places a **shared lock** on that row.

Now:

- ✓ Other transactions can also read it
- ✗ No transaction can update/delete it

Shared Lock Compatibility

Existing Lock	New Shared Lock	New Exclusive Lock
Shared Lock	✅ Allowed	❌ Blocked

◆ Exclusive Lock (X Lock)

An **exclusive lock** allows:

- Only one transaction to access and modify the data
- Prevents both:
 - Other reads (in some DB/isolation levels)
 - Other writes

👉 Used for **INSERT / UPDATE / DELETE**

Example

Transaction T1:

UPDATE account

SET balance = balance - 500

WHERE id = 1;

Database places an **exclusive lock**.

Now:

✗ Other transactions cannot read/write that row (depending on isolation level)

until transaction completes.

Exclusive Lock Compatibility		
Existing Lock	New Shared Lock	New Exclusive Lock
Exclusive Lock	✗ Blocked	✗ Blocked

3. What are the Isolation Level present?			
Isolation Level	Dirty Read Possible	Non-Repeatable Read Possible	Phantom Read Possible
Read Uncommitted	Yes	Yes	Yes
Read Committed	No	Yes	Yes
Repeatable Read	No	No	Yes
Serializable	No	No	No

Consistency High ↑

↓ Consistency Low

Dirty Read =>

A **dirty read** happens when one transaction reads data that was modified by another transaction **but not yet committed**.

If the first transaction later rolls back, the second transaction has read invalid or “dirty” data.

Problem Here ✗ [In below given example]

At **T3**, Transaction B read data (Booked) that was **not committed** by Transaction A.

Later at **T5**, Transaction A rolled back.

So Transaction B worked with invalid data.

This is called a **Dirty Read**.

Dirty Read Example

Time	Transaction A	Transaction B	Database State
T1	BEGIN TRANSACTION	BEGIN TRANSACTION	ID=1, Status=Free
T2	UPDATE row ID=1 → Status=Booked (<i>Not committed yet</i>)		ID=1, Status=Booked (<i>Uncommitted</i>)
T3		READ row ID=1 → gets Booked	ID=1, Status=Booked (<i>Still uncommitted</i>)
T4	Some issue occurs	Performs some computation using Booked status	ID=1, Status=Booked (<i>Still uncommitted</i>)
T5	ROLLBACK	COMMIT	ID=1, Status=Free

Non-Repeatable Read =>

Time	Transaction A	Transaction B	Database State
T1	BEGIN TRANSACTION	BEGIN TRANSACTION	ID=1, Balance=1000
T2	READ row ID=1 → Balance=1000		ID=1, Balance=1000
T3		UPDATE row ID=1 → Balance=2000	
T4		COMMIT	ID=1, Balance=2000
T5	READ row ID=1 again → Balance=2000		ID=1, Balance=2000
T6	COMMIT		ID=1, Balance=2000

Problem Here ✖

Transaction A read the same row twice:

- First read → 1000
- Second read → 2000

The row value changed during the same transaction because Transaction B updated and committed the data in between.

This is called a **Non-Repeatable Read**.

Phantom Read =>

Time	Transaction A	Transaction B	Database State
T1	BEGIN TRANSACTION	BEGIN TRANSACTION	Orders count where status='NEW' = 2
T2	SELECT * FROM orders WHERE status='NEW' → returns 2 rows		2 matching rows
T3		INSERT INTO orders (status='NEW')	
T4		COMMIT	Orders count where status='NEW' = 3
T5	SELECT * FROM orders WHERE status='NEW' again → returns 3 rows		3 matching rows
T6	COMMIT		3 matching rows

Problem Here ✗

Transaction A executed the same query twice:

```
SELECT * FROM orders WHERE status='NEW';
```

Results changed:

- First query → 2 rows
- Second query → 3 rows

A new row appeared during the transaction.

This extra row is called a **Phantom Row**. So this issue is called a **Phantom Read**.

Isolation Level Behavior			
Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read
Read Uncommitted	✓ Possible	✓ Possible	✓ Possible
Read Committed	✗ Prevented	✓ Possible	✓ Possible
Repeatable Read	✗ Prevented	✗ Prevented	✓ Possible
Serializable	✗ Prevented	✗ Prevented	✗ Prevented

ISOLATION LEVEL	Locking Strategy
<i>Read Uncommitted</i>	Read : No Lock acquired Write: No Lock acquired
<i>Read Committed</i>	Read: Shared Lock acquired and Released as soon as Read is done Write: Exclusive Lock acquired and keep till the end of the transaction
<i>Repeatable Read</i>	Read: Shared Lock acquired and Released only at the end of the Transaction Write: Exclusive Lock acquired and Released only at the end of the Transaction
<i>Serializable</i>	Same as Repeatable Read Locking Strategy + apply Range Lock and lock is release only at the end of the Transaction.

You can set the **transaction isolation level** either:

1. Directly in SQL (database level)
2. In application code (like Spring Framework using @Transactional)

◆ 1. Using SQL

Syntax

```
SET TRANSACTION ISOLATION LEVEL <LEVEL>;
```

Example: Read Committed

```
SET TRANSACTION ISOLATION LEVEL READ COMMITTED;
```

```
BEGIN TRANSACTION;
```

```
SELECT * FROM account;
```

```
COMMIT;
```

```

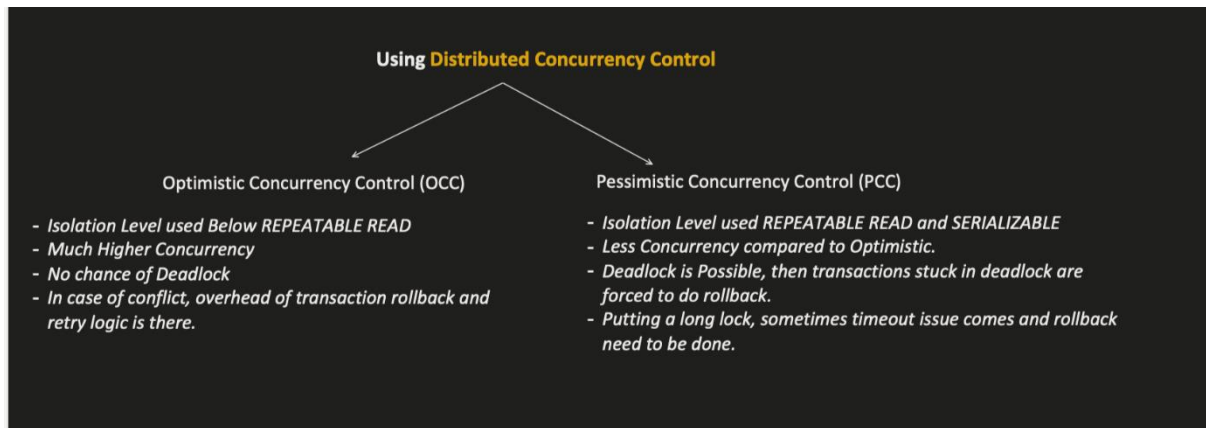
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

BEGIN_TRANSACTION;

SELECT Query;
Update Query;
If Success:
    COMMIT TRANSACTION;
else:
    ROLLBACK TRANSACTION;

END_TRANSACTION

```



Optimistic Concurrency Control (OCC)

Optimistic Concurrency Control assumes:

Conflicts between transactions are rare, so transactions can proceed without locking initially.

Instead of locking rows early, OCC checks for conflicts **at the time of update/commit** using a **version number** or timestamp.

Isolation levels used **READ UNCOMMITTED** and **READ COMMITTED**

Used heavily in:

- High-concurrency systems
- Microservices
- Distributed systems
- Web applications

◆ Main Idea

Each row contains a version field.

Example:

ID Status Version

1 Free 1

When updating:

- Transaction validates version before commit
- If version changed → another transaction already modified the row
- Current transaction fails and retries/rolls back

OCC Flow Example

Time	Transaction A	Transaction B	Database State
T1	BEGIN TRANSACTION	BEGIN TRANSACTION	ID=1, Status=Free, Version=1
T2	READ row ID=1 (Version=1)	READ row ID=1 (Version=1)	ID=1, Status=Free, Version=1
T3	Validate Version=1 before update		ID=1, Status=Free, Version=1
T4	UPDATE Status=Booked, Version=2		ID=1, Status=Booked, Version=2
T5	COMMIT		ID=1, Status=Booked, Version=2
T6		Validate Version=1 before update	ID=1, Status=Booked, Version=2
T7		Version mismatch detected → ROLLBACK	ID=1, Status=Booked, Version=2

🔥 What Happened Here?

Both transactions read:

Version = 1

📄

Transaction A updated first:

Version becomes 2

📄

Now Transaction B tries to update using old version:

WHERE version = 1

📄

But DB row already has:

Version = 2

📄

So update fails.

This prevents:

- Lost updates
- Data overwrite issues



◆ Advantages ✓

Benefit	Explanation
High concurrency	No long-term locking
Better performance	Reads are non-blocking
No deadlocks	Since locks are minimal
Good for read-heavy systems	Many readers, few conflicts

◆ Disadvantages ✗

Problem	Explanation
Retry logic needed	Failed transactions must retry
Not good for frequent conflicts	Many rollbacks can happen
Extra version column	Required in tables

Pessimistic Concurrency Control (PCC)

Pessimistic Concurrency Control assumes:

Conflicts between transactions are likely, so data should be locked immediately before modification.

Instead of allowing parallel updates and checking later, PCC locks the row early so other transactions must wait.

Used in:

- Banking systems
- Payment systems
- Ticket booking
- Inventory management

where strong consistency is critical.

◆ Main Idea

When a transaction wants to update data:

- Database places an **Exclusive Lock**

- Other transactions cannot modify the same row until lock is released

This prevents:

- Lost updates
- Dirty writes
- Concurrent modification issues

PCC Flow Example

Time	Transaction A	Transaction B	Database State
T1	BEGIN TRANSACTION	BEGIN TRANSACTION	ID=1, Status=Free
T2	SELECT row ID=1 FOR UPDATE		ID=1 Locked by Txn A
T3	UPDATE Status=Booked		ID=1, Status=Booked (<i>Locked by Txn A</i>)
T4		SELECT row ID=1 FOR UPDATE	Transaction B waits
T5	COMMIT		Lock released
T6		Lock acquired after wait	ID=1, Status=Booked
T7		Reads latest committed value	ID=1, Status=Booked
T8		ROLLBACK / decides no update	ID=1, Status=Booked

🔥 What Happened Here?

Transaction A locked the row first:

```
SQL
SELECT * FROM ticket
WHERE id = 1
FOR UPDATE;
```

Now Transaction B tries accessing same row.

But:

- row already locked
- B must wait

Only after Transaction A commits:

- lock releases
- Transaction B proceeds

This guarantees consistency.

◆ SQL Example

Lock Row for Update

```
SELECT *  
FROM ticket  
WHERE id = 1  
FOR UPDATE;
```

This creates:

- Exclusive lock
- Prevents concurrent updates

◆ In Spring Boot / JPA

Using pessimistic lock mode:

```
@Lock(LockModeType.PESSIMISTIC_WRITE)  
@Query("SELECT t FROM Ticket t WHERE t.id = :id")  
Ticket findTicketForUpdate(Long id);
```

◆ Advantages ✓

Benefit	Explanation
Strong consistency	Prevents concurrent modifications
No retry logic needed	Transactions wait instead
Good for high-conflict systems	Safer for critical updates

◆ Disadvantages ✗

Problem	Explanation
Lower concurrency	Transactions wait
Deadlocks possible	Two transactions may wait forever
Performance overhead	Long locks reduce throughput
Timeout issues	Waiting transactions may fail

◆ Deadlock Example

Transaction A locks Row1

Transaction B locks Row2

Now:

- A waits for Row2
- B waits for Row1

Both stuck forever → deadlock.

Database detects this and rolls back one transaction.

◆ OCC vs PCC		
Feature	Optimistic (OCC)	Pessimistic (PCC)
Locking	Minimal	Immediate
Conflict Handling	Retry on conflict	Wait/block
Concurrency	High	Lower
Deadlock	Rare	Possible
Performance	Better	Slower
Best For	Read-heavy systems	Write-heavy systems